

Playwright API Automation Framework

Technical Guide

Reviewed against the repository source on 30 September 2026.

1. What this project does

This repository is a TypeScript API test framework built on Playwright Test. It exercises the Restful Booker style authentication and booking endpoints through reusable API client classes. The tests cover token generation and invalid credentials, booking creation, retrieval of a missing booking, and a complete create/update/partial-update/delete lifecycle. Payloads and successful responses are checked with JSON Schema; separate contracts assert endpoint status, response shape, and selected headers.

The tests target an external API configured by environment variables. They do not start an API server. Docker packages the Node/Playwright test runner; GitHub Actions builds and runs that image and collects test reports.

2. Architecture and request lifecycle

The flow is: test spec → custom Playwright fixture → BookingApi/AuthApi → BaseApi → Playwright APIRequestContext → configured service. Factories produce test credentials and varied booking data. Models provide TypeScript shapes. Schemas validate payloads and responses; contracts validate response expectations; assertion helpers compare business fields. The BaseApi logs and attaches request/response data, parses JSON (falling back to text), validates configured schemas, handles unexpected HTTP errors, and retries one authenticated request after a 401 by clearing cached authentication.

AuthManager caches one token in process memory for 30 minutes. Protected PUT, PATCH, and DELETE methods obtain a token and send it as ``Cookie: token=...``. GET and POST do not require authentication by default. The API-specific methods add their endpoint path and schema options.

3. Repository map

- ``api/``: shared request implementation (``BaseApi``) and endpoint clients (``AuthApi``, ``BookingApi``).
- ``config/envConfig.ts``: loads environment settings and exports base URL and credentials.
- ``constants/apiEndpoints.ts``: joins the base URL to ``/auth`` and ``/booking``.
- ``fixtures/apiFixture.ts``: creates API clients around Playwright's request fixture and adds an Allure environment label.
- ``tests/api/``: authentication, booking creation, retrieval, and lifecycle tests.
- ``helpers/``: schema/contract/request validation, assertions, error formatting, logging, and report attachments.
- ``src/factories/``: valid and invalid authentication/booking payload builders; booking values use Faker.
- ``src/schemas/``: JSON Schemas for requests, responses, and error responses.
- ``src/contracts/``: expected API statuses, response schemas, and response header requirements.
- ``models/``: shared TypeScript interfaces such as Booking, Auth, RequestOptions, and ApiResult.
- ``scripts/reportGenerator.ts``: combines Playwright JSON reports into a summary, email text, and PDF.
- ``utils/allureEnvironment.ts``: Allure environment utility.

- ``Dockerfile``, ``docker-compose.yml``: containerized test execution and report volume mounts.
- ``.github/workflows/docker-tests.yml``: scheduled, push, pull-request, and manual CI pipeline.
- ``playwright.config.ts``: test discovery, environment loading, browser project, retries, and reporters.

Generated output directories include ``reports/``, ``allure-results/``, ``playwright-report/``, and ``test-results/``; these are runtime artifacts, not source modules.

4. Configuration and credentials

``playwright.config.ts`` chooses ``TEST_ENV`` (defaults to ``qa``) and attempts to load ``environments/<name>.env`` without overwriting variables already in the process. It uses ``BASE_URL`` as Playwright's base URL. ``config/envConfig.ts`` separately loads the same file and exports ``BASE_URL``, ``BOOKER_USERNAME``, and ``BOOKER_PASSWORD``; endpoint constants depend on this configuration at import time.

Set these variables in the process environment or a matching environment file:

```
BASE_URL=https://restful-booker.herokuapp.com
BOOKER_USERNAME=admin
BOOKER_PASSWORD=password123
```

The checked-in tree reviewed here does not contain an ``environments/`` directory. Create the appropriate local environment file when needed, or inject variables in the shell/CI. Do not commit real credentials. CI references repository secrets named ``BASE_URL``, ``BOOKER_USERNAME``, and ``BOOKER_PASSWORD``. Docker Compose forwards these values and mounts report folders back to the host.

5. Install and run

Requirements: Node.js/npm and Docker for container runs. The lockfile and scripts define the dependencies and commands.

```
npm ci
npm run test:qa
```

Other package scripts:

- ``npm test``: run Playwright using current environment settings.
- ``npm run test:staging``: set ``TEST_ENV=staging`` and run tests.
- ``npm run test:smoke``: run QA tests whose title contains ``@smoke``.
- ``npm run test:regression``: run QA tests whose title contains ``@regression``.
- ``npm run test:report:qa``, ``test:report:smoke``, ``test:report:regression``: run QA tests with line and JSON reporters to their configured report filenames.
- ``npm run generate-report``: aggregate JSON files under ``reports/`` into summary artifacts.
- ``npm run allure:generate`` / ``npm run allure:open``: generate or open an Allure report from ``allure-results``.

Run one spec with ``npx playwright test tests/api/createBooking.spec.ts``; filter titles with ``npx playwright test --grep @smoke``. The config runs headless Chromium, uses up to four workers locally, one worker in CI, and retries twice in CI. On a first retry it records a trace; screenshots are captured only on failure. Video is configured on. The expectation timeout is four minutes.

6. Tests and coverage

- ``auth.spec.ts``: valid credentials produce a nonempty token and satisfy the success contract; invalid credentials are expected to return HTTP 200 with ``reason: Bad credentials`` and satisfy the error contract.

- ``createBooking.spec.ts``: generates and validates a booking, creates it, checks returned ID and booking fields, then retrieves and compares it. Regression cases verify local schema rejection of invalid price, invalid ``depositpaid``, and an unknown field.
- ``getBooking.spec.ts``: the positive retrieval example is commented out; the active test requests a deliberately missing ID and expects 404 with the error schema.
- ``updateBooking.spec.ts``: creates a booking, replaces it with PUT, verifies with GET, changes names with PATCH and checks unchanged fields remain, deletes it, then expects a 404 on retrieval. It also verifies an unauthenticated update receives 403.

The active test tags determine the smoke/regression selections. At present the active smoke-tagged tests are successful authentication and booking creation. The main CRUD lifecycle and negative cases are regression-tagged.

7. Validation and error behavior

``RequestOptions`` can specify ``requiresAuth``, ``expectedStatus``, ``requestSchema``, and ``responseSchema``. AJV validates schemas. Contract validation checks the expected status and optional response headers. ``BaseApi`` throws a detailed error for a non-2xx result unless the returned status matches ``expectedStatus``; an expected error response can therefore be asserted by the test. For such cases, provide the appropriate error response schema because endpoint clients may set a success schema by default. ``BookingFactory`` generates a future check-in and later checkout date, random identity/price/deposit/needs, and helper variants for negative cases.

8. Reports and artifacts

Playwright config enables HTML (``playwright-report/``), list, JSON (``reports/<REPORT_FILE or json-test-report.json>``), JUnit XML (``json-test-report.xml`` by default), and Allure output. Open the HTML report with ``npx playwright show-report playwright-report``. Allure commands are exposed through npm scripts.

``scripts/reportGenerator.ts`` searches recursively for JSON files in ``reports/``, counts final test results, and writes ``reports/summary.json``, ``reports/email.txt``, ``reports/email-subject.txt``, and ``reports/API_Test_Report.pdf``. It expects report JSON files in the Playwright JSON reporter format. Supply those input files first; the generator exits if the reports folder or JSON inputs are missing.

9. Docker and CI

The Dockerfile uses the Playwright Noble image, copies the package manifests, installs with ``npm ci``, copies the project, and defaults to ``npx playwright test``. Compose service ``api-tests`` injects environment/report variables, runs optional ``TEST_ARGS``, and mounts report directories for host access. Typical invocation:

```
$env:BASE_URL = 'https://restful-booker.herokuapp.com'
$env:BOOKER_USERNAME = 'admin'
$env:BOOKER_PASSWORD = 'password123'
$env:TEST_ENV = 'qa'
docker compose run --rm -e TEST_ARGS='--grep @smoke' api-tests
```

The workflow ``github/workflows/docker-tests.yml`` runs smoke, regression, and QA/staging matrix jobs. It is triggered by selected pushes, pull requests to ``main``, a daily schedule, or manual dispatch. Jobs build Docker images, inject GitHub secrets, run filtered or full suites, and upload results. A downstream report job downloads those artifacts, attempts report aggregation/Allure generation, uploads a final artifact, publishes Allure to ``gh-pages``, and sends an email using configured secrets.

10. Implementation notes for maintainers

- Keep credential names consistent: the code and workflow use `BOOKER\_USERNAME` / `BOOKER\_PASSWORD`; README sections that mention `USERNAME` / `PASSWORD` do not match the runtime implementation.
- Current source tree has no `environments/` directory even though configuration attempts to load files there. Environment variables are a valid alternative.
- Reporter output is configured in `playwright.config.ts`; the checked-in README refers to different folders/spec paths in places.
- In CI, check that each configured secret exists and that report artifact paths match the files generated by the reporters. The workflow references extra report scripts/paths; inspect those assumptions when maintaining CI.
- Authentication tests validate invalid credentials as HTTP 200 with a reason body, whereas the shared BaseApi only throws automatically for non-2xx responses. Keep tests and the target API's documented behavior aligned.
- GET booking's positive test is currently commented out, so the active spec only checks the missing-ID case.

11. Quick troubleshooting

- **Missing base URL or credentials:** export the three `BOOKER\_\*`/`BASE\_URL` variables or create the selected `environments/<TEST\_ENV>.env` file.
- **401 on a protected operation:** check credentials, API availability, and cookie-token behavior; AuthManager refreshes once after a 401.
- **Schema validation error:** inspect the failing field and AJV error details; request payload schemas reject malformed/extra fields before a network call.
- **No consolidated PDF report:** ensure Playwright JSON reports exist under `reports/` before invoking `npm run generate-report`.
- **Docker output absent on host:** confirm Compose volume mounts and that the container can create the report directories.

12. Key files

Start with `playwright.config.ts`, `fixtures/apiFixture.ts`, `api/BaseApi.ts`, `api/BookingApi.ts`, `api/AuthApi.ts`, and `tests/api/`. For payload rules inspect `src/schemas/`; for business-level response guarantees inspect `src/contracts/`; for CI and containers inspect `.github/workflows/docker-tests.yml`, `Dockerfile`, and `docker-compose.yml`.

